Polymorphism in C++

1. What is Polymorphism?

The word polymorphism simply means "many forms".

In C++, it means that one object can behave in different ways depending on the situation.

> *Real-life example: A man at home is a father. The same man at work is an employee. At a wedding, he is a husband. He is the same person — but he plays different roles in different places. That is exactly what polymorphism is — one thing, many behaviors.*

2. Why Should You Use Polymorphism?

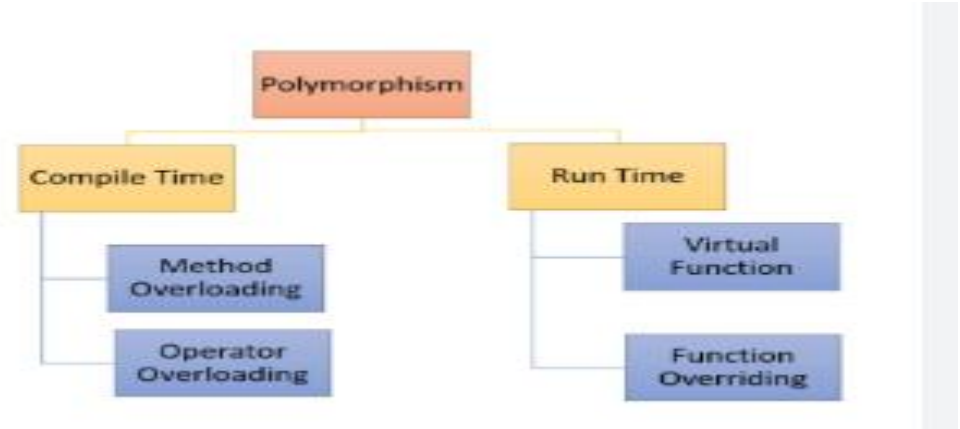
Here is a simple scenario:

- You want to calculate the area of a circle and the area of a rectangle.
- Without polymorphism, you would write two separate functions with two different names.
- With polymorphism, you write one function called area() and let each shape handle it differently.

This makes your code:

- **Cleaner — less repetition**
- **Consistent — same function name for similar tasks**
- **Easier to maintain — change in one place, not everywhere**

3. Two Types of Polymorphism



Rules for Inheritance and Polymorphism:

Rule 1: Data Members

◦ Derived class inherits all data members of Base class

◦ Derived class may add data members of its own

Rule 2: Member Functions

◦ Derived class inherits all member functions of Base class

◦ Derived class may **override/ replaces** a member function of Base class by redefining it with the same signature

◦ Derived class may **overload or add a member function** of Base class by redefining it with the same name; but different signature

◦ Derived class may add new member functions

Rule 3: Access Specification

◦ Derived class cannot access private members of Base class

◦ Derived class can access protected members of Base class

Rule 4: Constructor -Destructor

◦ A constructor of the Derived class must first call a constructor of the Base class to construct the Base class instance of the Derived class

◦ The destructor of the Derived class must call the destructor of the Base class to destruct the Base class instance of the Derived class

Override

- **Override** means the child class **replaces** the parent's function with its own version.
- Rule: The function name **and** the input/output (the *signature*) must be exactly the same.

4. Compile-Time Polymorphism

The compiler decides at compile time which function to call. It is fast but less flexible.
Think of it as making all decisions before the program runs.

4A — Function Overloading

Same function name, different parameters.

You can have multiple functions with the same name as long as they take different types or numbers of arguments.

Overload

- **Overload** means the child class adds **another version** of the function with the same name but **different inputs**.
- Rule: Same function name, but different *signature* (like different parameters).

  Simple program for overloading:

```
class Animal {
public:
   void sound()
{
cout << "Some sound"; }
};

class Dog : public Animal
{
public:
   void sound(int times)
{
      for(int i=0; i<times; i++)
                  cout << "Bark ";
   }  // Overloads sound with an extra parameter
};
```

```
void myFunc(int a)           // called when you pass an integer
void myFunc(double a)        // called when you pass a decimal
void myFunc(int a, int b)    // called when you pass two integers
```

When you call myFunc(10) — the compiler sees an integer and picks the first one. Simple!

Key rule: Same name + Different arguments = Function Overloading. No inheritance needed!

Program :1 Write a C++ program for Function Overloading in compile time polymorphism

```
#include <bits/stdc++.h>
using namespace std;
class functionOverloadingExample
{
   public:
   // function with 1 int parameter
   void myFunc(int a)
   {
      cout << "a = " << a << "\n\n";
   }
   // myFunction with same name but 1 double parameter
   void myFunc(double a)
   {
      cout << "a =  " << a << "\n\n";
   }
   // myFunction with same name and 2 int parameters
   void myFunc(int a, int b)
   {
      cout << "a = " << a << ", b = " << b << "\n\n";
   }
};
int main()
 {
   functionOverloadingExample obj;
   // Which myFunction is called will depend on the parameters passed
   // The first 'myFunc' is called
   obj.myFunc(10);
   // The second 'myFunc' is called
```

```
    obj.myFunc(10.20);
    // The third 'myFunc' is called
    obj.myFunc(100, 200);
    return 0;
}
```

What is the output?

4B — Operator Overloading

Operator overloading means teaching math symbols like +, -, *, or == to work with your own special objects, not just numbers.

Analogy

Imagine you have two toy boxes:

- Normally, + means "add numbers" (like 2 + 3 = 5).
- But if you say: "Box1 + Box2," you want it to mean "combine the toys inside both boxes."

Operator overloading lets you **teach the computer** what + should do for toy boxes, or any custom thing you make.

Operators you CANNOT overload:

- . (dot operator)
- :: (scope operator)
- ?: (ternary operator)
- Sizeof

Program :2 Write a C++ program to explain the operator overloading in compile time polymorphism

```
#include <iostream>
using namespace std;

class Number
{
private:
    int value;
```

```
public:
    // Constructor initializes value to 8
    Number() : value(8)
{
}

    // Overload the ++ operator (prefix form)
    void operator ++()
{
        value = value + 20;  // Add 20 instead of 1
    }

    // Display function
    void display() {
        cout << "The updated value is: " << value << endl;
    }
};

int main() {
    Number num;   // value starts at 8

    ++num;        // Overloaded ++ adds 20

    num.display(); // Output: The updated value is: 28

    return 0;
}
```

Explanation:

- Class Number-> Has one private variable value.
- Starts at 8 when you create an object.
- Overloaded ++ Operator
- Normally, ++ adds 1 to numbers.Here, we changed it so that ++ adds 20 instead.
- This is called operator overloading teaching the operator a new trick.
- Main Function

- Creates num with value = 8.
- ++num; → increases value by 20 → now value = 28.
- num.display(); → prints:

Output:

The updated value is: 28

5. Run-Time Polymorphism

Here, the decision of which function to call is made while the program is actually running.
It is slightly slower but much more flexible and powerful for complex programs.

5A — Function Overriding

Wat is Function over riding

Same function name in both parent (base) and child (derived) class.
The child class gives its own version of the function. When the child object calls that function — the child's version runs, not the parent's.

Rules:

- Same function name
- Same arguments
- Same return type
- Inheritance is REQUIRED

Note: What is overriding?

- **Override** means the child class **replaces** the parent's function with its own version.
- Rule: The function name **and** the input/output (the *signature*) must be exactly the same.

Simple Program to understand Overriding

```
Class Animal
{
public:
void sound() { cout << "Some sound"; }
};
class Dog : public Animal {
public:
void sound() { cout << "Bark"; }  // Overrides Animal's sound
};
```

When you create a Parrot object and call display(), it prints "I am a parrot" — not "I am a bird".

Program :3 Write a C++ program to explain the function overriding in runtime polymorphism

```
#include <iostream>
using namespace std;

// Base class
class Bird {
public:
    void display() {
        cout << "I am the display function of the base class" << endl;
    }
};

// Derived class
class Parrot : public Bird {
public:
    // Overrides the display() function of Bird
    void display() {
        cout << "I am the display function of the derived class" << endl;
    }
};

int main() {
    Bird b;     // Object of base class
    Parrot p;   // Object of derived class

    b.display();  // Calls base class function
    p.display();  // Calls derived class function

    return 0;
```

```
}
```

Output:

I am the display function of the base class

I am the display function of the derived class

5B — Virtual Functions

A virtual function is declared in the base class using the keyword 'virtual'. When you use a base class pointer to point to a child object and call a virtual function — the child's version gets called at runtime.

A **virtual function** is a special kind of function in a base class that says: *"Hey, if a child class wants to replace me, use the child's version at run time."*

It's like giving the child class the **final say** when the program is running.

Program :4 Write a C++ program to explain the function overriding using Virtual function in runtime polymorphism

```
class Bird {
public:
    virtual void display()
{
 cout << "Bird display";
}  // virtual!
    void print()
 {
cout << "Bird print";
}    // NOT virtual
};

class Parrot : public Bird {
public:
    void display()
{
cout << "Parrot display";
}
    void print()
```

```
{
cout << "Parrot print";
 }
};

   int main()
{
   Bird* b;       // Pointer to base class
   Parrot p;      // Object of derived class
   b = &p;        // Base pointer points to Parrot object

   b->sound();    // Calls Parrot's version because of 'virtual'

   return 0;
}
}
```

Key points about virtual functions:

- Always declared in the base class using the 'virtual' keyword
- Always overridden in the child class
- Called during run time
- Accessed through a pointer or reference to the base class

What is an Abstract Base Class?

- An **abstract base class** is like a **blueprint** that cannot be used directly.
- It has at least one **pure virtual function** (a function with no body).
- Child classes must **fill in the details** of that function.

👉 In short: It's a "template" for other classes.
The abstract base class sets the rule, and the child classes decide how to follow it.

Write a C++ program to explain a Abstract base Class.

```
#include <iostream>
using namespace std;

// Abstract base class
class Bird
{
```

```
public:
    // Pure virtual function (no body)
    virtual void sound() = 0;
};

// Derived class
class Parrot : public Bird {
public:
    void sound()
{
        cout << "Parrot says: Hello!" << endl;
    }
};

// Another derived class
class Crow : public Bird {
public:
    void sound()
 {
        cout << "Crow says: Caw Caw!" << endl;
    }
};

int main() {
    Bird* b;       // Base class pointer

    Parrot p;
    Crow c;

    b = &p;
    b->sound();    // Calls Parrot's Child Class  version

    b = &c;
    b->sound();    // Calls Crow's Child Class version

    return 0;
}
```

6. Quick Comparison Table

Feature	Compile-Time	Run-Time
Also called	Static / Early Binding	Dynamic / Late Binding
When resolved?	During compilation	During execution (runtime)
How implemented?	Function & Operator Overloading	Function Overriding & Virtual Functions
Speed	Faster	Slightly slower

Flexibility	Less flexible	More flexible
Best for	Simple, straightforward tasks	Complex, real-world problems
Inheritance needed?	No	Yes

7. Exam hints

Concept	One-Line Memory Trick
Polymorphism	One name, many behaviors
Function Overloading	Same name, different inputs — compiler decides
Operator Overloading	Teach +, ++ etc. to work on your own objects
Function Overriding	Child rewrites parent's function
Virtual Function	Base pointer → child function runs at runtime

Inheritance in C++: Types and Examples

What is inheritance?

- Inheritance is a fundamental concept in C++ that enables a class to inherit/use properties and behaviors from another class,
- Inheritance promoting code reuse and hierarchy within programs.

Syntax of Inheritance

In C++, inheritance syntax involves creating a derived class that inherits attributes and behaviors from a base class. Here's a basic representation:

```
// Base Class
class Base {
public:
void display()
 {
```

```
cout << "This is from the base class." << endl;
}
};

// Derived Class inheriting from Base
class Derived : public Base {
public:
void show()
 {
cout << "This is from the derived class." << endl;
}
};
```

What are the rules of Inheritance:

Rule 1: Data Members

◦ Derived class inherits all data members of Base class

◦ Derived class may add data members of its own

Rule 2: Member Functions

◦ Derived class inherits all member functions of Base class

◦ Derived class may **override/ replaces** a member function of Base class by redefining it with the same signature

◦ Derived class may **overload or add a member function** of Base class by redefining it with the same name; but different signature

◦ Derived class may add new member functions

Rule 3: Access Specification

◦ Derived class cannot access private members of Base class

◦ Derived class can access protected members of Base class

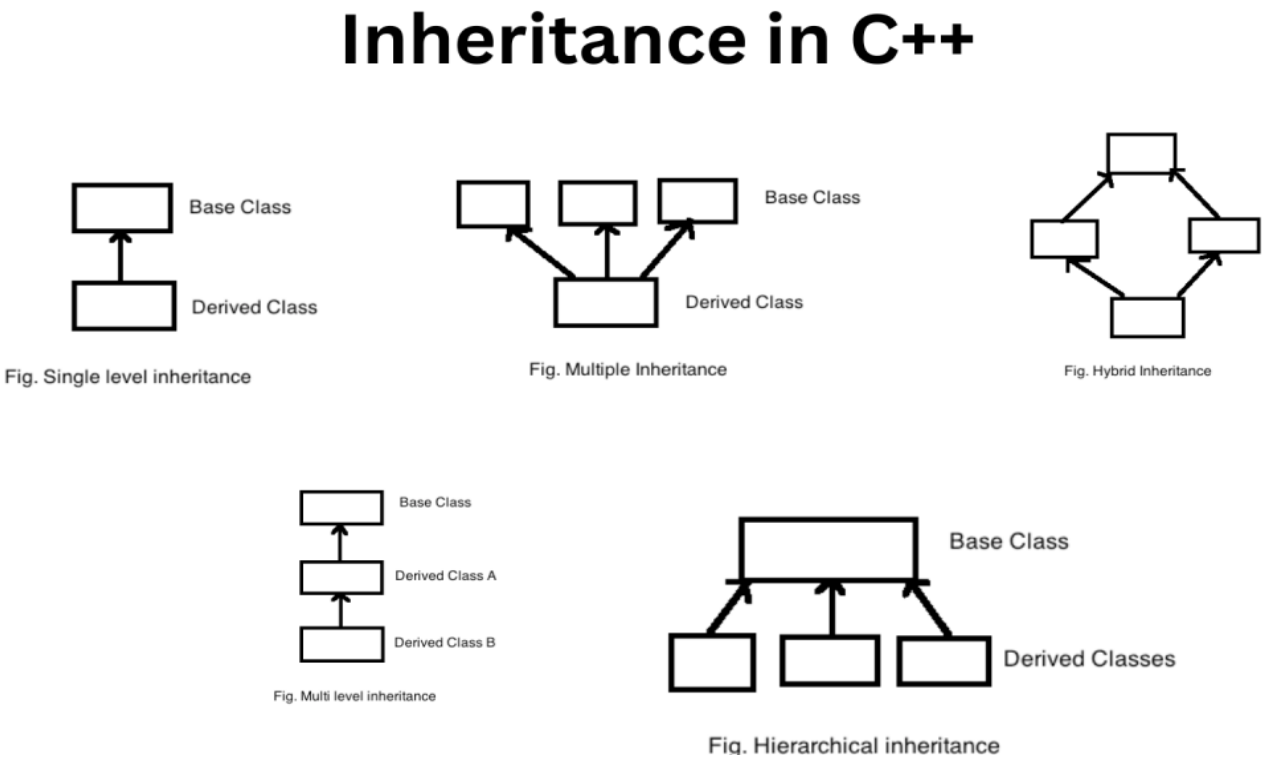
Rule 4: Constructor -Destructor

◦ A constructor of the Derived class must first call a constructor of the Base class to construct the Base class instance of the Derived class

◦ The destructor of the Derived class must call the destructor of the Base class to destruct the Base class instance of the Derived class

Types of Inheritance

- **Single Level Inheritance**
- **Multi-Level Inheritance**
- **Hierarchical Inheritance**
- **Multiple Inheritance**
- **Hybrid Inheritance**



1. Single Inheritance

Single Level Inheritance involves deriving only one class from the base class. It establishes a straightforward parent-child relationship between two classes.

```
      #include <iostream>
using namespace std;
```

```
class Base
{
public:
    void display()
{
        cout <<"This is the Base class" << endl;
    }
};

class Derived : public Base {
public:
    void show()
  {
        cout << "This is the Derived class" << endl;
    }
};

int main() {
    Derived d;
    d.display(); // Accessing Base class function from
    d.show();    // Accessing Derived class function

    return 0;
}
```

- **Output:**

This is the Base class
This is the Derived class

2. Multi-Level Inheritance

Multi-Level Inheritance signifies a chain of inheritance where one derived class becomes the base class for another derived class. It allows for multiple levels of derived classes.

```
#include <iostream>
using namespace std;

class A
{
public:
   void displayA()
{
      cout << "This is class A" << endl;
   }
};

class B : public A
{
public:
   void displayB() {
      cout << "This is class B" << endl;
   }
};

class C : public B
{
public:
   void displayC() {
      cout << "This is class C" << endl;
   }
};

int main() {
   C c;
   c.displayA(); // Accessing Class A method
```

```
    c.displayB(); // Accessing Class B method
    c.displayC(); // Accessing Class C method

    return 0;
}
```

Output:

```
This is class A
This is class B
This is class C
```

3. Hierarchical Inheritance

Hierarchical Inheritance occurs when multiple classes are derived from a single base class. It forms a hierarchical structure, branching out from a common base.

```
#include <iostream>
using namespace std;

class Base
{
public:
    void display()
{
        cout << "This is the Base class" << endl;
    }
};

class Derived1 : public Base
{
public:
    void show1()
```

```
{
        cout << "This is the first derived class" << endl;
    }
};

class Derived2 : public Base {
public:
    void show2() {
        cout << "This is the second derived class" << endl;
    }
};

int main() {
    Derived1 d1;
    Derived2 d2;
    d1.display(); // Accessing Base class function via first derived class
    d1.show1();   // Accessing first derived class function
    d2.display(); // Accessing Base class function via second derived
class
    d2.show2();   // Accessing second derived class function

    return 0;
}
```

Output:

```
This is the Base class
This is the first derived class
This is the Base class
This is the second derived class
```

4. Multiple Inheritance

Multiple Inheritance occurs when a single class inherits properties and methods from multiple base classes. It provides a way to combine functionalities from different sources.

```
#include <iostream>
using namespace std;

class A {
public:
    void displayA() {
        cout << "This is class A" << endl;
    }
};

class B {
public:
    void displayB() {
        cout << "This is class B" << endl;
    }
};

class C : public A, public B {
public:
    void displayC() {
        cout << "This is class C" << endl;
    }
};

int main() {
    C c;
    c.displayA(); // Accessing Class A method
    c.displayB(); // Accessing Class B method
```

```
    c.displayC(); // Accessing Class C method

    return 0;
}
```

Output:

```
This is class A
This is class B
This is class C
```

Hybrid Inheritance

Hybrid Inheritance is a combination of multiple inheritance types, encompassing multilevel, multiple, and hierarchical inheritance. It brings together various inheritance forms.

```
#include <iostream>
using namespace std;
class A {
public:
    void displayA() {
        cout << "This is class A" << endl;
    }
};

class B : public A {
public:
    void displayB() {
        cout << "This is class B" << endl;
    }
};

class C : public A {
public:
```

```
    void displayC() {
        cout << "This is class C" << endl;
    }
};

class D : public B, public C {
public:
    void displayD() {
        cout << "This is class D" << endl;
    }
};

int main() {
    D d;
    d.displayA(); // Accessing Class A method
    d.displayB(); // Accessing Class B method
    d.displayC(); // Accessing Class C method
    d.displayD(); // Accessing Class D method

    return 0;
}
```

Output:

```
This is class A
This is class B
This is class A
This is class C
This is class D
```

Constructors and Destructors in Inheritance

Rule:

- When a Derived object is created, the Base constructor runs first, then the Derived constructor.
- When the object is destroyed, the Derived destructor runs first, then the Base destructor.
- This order ensures base parts are initialized before derived parts and cleaned up after derived parts.

Write a C++ program to explain constructor and Desturctor inheritance

```
#include <iostream>
using namespace std;

class Base {
public:
   Base()  { cout << "Base constructor\n"; }
   ~Base() { cout << "Base destructor\n"; }
};

class Derived : public Base {
public:
   Derived()  { cout << "Derived constructor\n"; }
   ~Derived() { cout << "Derived destructor\n"; }
};

int main() {
   cout << "Create local Derived object:\n";
```

```
    Derived d;          // constructors run here
    cout << "Leaving main block\n";
    return 0;           // destructors run when d goes out of scope
}
```

Output: (Read the Rule)

Create local Derived object:
Base constructor
Derived constructor
Leaving main block
Derived destructor
Base destructor